

Efficient and Parallel Implementation of Isogeny-based Deterministic Group Actions

Weize Wang¹, Yi-Fu Lai², Kaizhan Lin¹✉ and Yunlei Zhao^{1,3}

¹ College of Computer Science and Artificial Intelligence, Fudan University, Shanghai, China

wzwang23@m.fudan.edu.cn

linkzh@fudan.edu.cn

ylzhao@fudan.edu.cn

² Shanghai Jiao Tong University, Shanghai, China

yifu.lai@sjtu.edu.cn

³ State Key Laboratory of Cryptology, Beijing, China

Abstract. Recent work by Houben (Asiacrypt’25) introduced a new formulation for class group actions on supersingular elliptic curves oriented by an imaginary quadratic order for an arbitrarily large discriminant. The algorithm is not only constant-time but also fully deterministic, dummy-free, and branch-free. As a result, it gives the fastest isogeny-based non-interactive key exchange (NIKE) in theory, referred to as OSIDH-LD in this paper. However, the current proof-of-concept SageMath implementation remains substantially slower than mainstream post-quantum key-exchange candidates. In this paper, we develop an efficient implementation of OSIDH-LD with several approaches. First, we provide algorithmic-level optimizations: (i) we develop the "tail pruning" approach such that key agreement avoids redundant orientation updates. This optimization maintains the fully deterministic and dummy-free feature of OSIDH-LD; (ii) we adapt a faster codomain isomorphism identification adapted from the technique used in the SQIsign implementations; and (iii) we present effective isogeny-computation strategies tailored to the cost profile of OSIDH-LD. Second, we adapt the parallelism technique. We apply the fork-join parallel execution model to optimize the class group action performance, and achieve a near perfect parallelism in key generation, and a better performance in key agreement.

We provide two kinds of implementations to show the impacts of our improvements. The first one is in C with assembly language for field arithmetic, which verifies the correctness of our optimization techniques targeting OSIDH-LD. The experimental results show that our techniques lead to an overall $1.56\times$ and $1.87\times$ acceleration for key generation and key agreement resp. On an Intel Core i7 CPU, the resulting costs are 12.8 Gcycs (KeyGen) and 10.57 Gcycs (KeyAgree) at a conjectured CSIDH-4096 security strength. Second, we provide parallel implementations that exploit multi-threads and AVX-512 vector extensions, respectively, by batching independent subroutines in the class group action. In particular, the AVX-512 vectorized implementation is $4.97\times$ faster than the improved C+assembly implementation in key generation, which is close to the theoretical optimum.

Keywords: Isogeny · Class Group Actions · Parallelism · Post-Quantum Cryptography

1 Introduction

Non-interactive key exchange (NIKE) enables two parties to derive a shared secret by only exchanging the public keys, without sending any additional public-key-dependent information. This “no extra message” property makes NIKE particularly attractive in

asynchronous settings and in 0-RTT style handshakes by saving both the bandwidth and round complexity, and it is widely used in the security protocols such as TLS and HTTPS.

Achieving NIKE in the post-quantum setting, however, is notoriously difficult. While many post-quantum proposals naturally instantiate KEM-style (interactive) key exchange, only very few isogeny-based or lattice-based constructions [CLM⁺18, GdKQ⁺24, Hou26] realize NIKE with plausible quantum resistance and feasible performance. Early isogeny-based NIKE can be traced back to the CRS key exchange [Cou06, RS06], an ordinary-curve class-group action construction. Roughly speaking, given a fixed starting curve E , the action of an element $\mathbf{a}$ in the commutative group defines an isogeny mapping E to E_A , denoted as $\mathbf{a} * E = E_A$. By exchanging the public keys $E_A = \mathbf{a} * E$ and $E_B = \mathbf{b} * E$, two parties can obtain the same share key $\mathbf{b} * E_A = \mathbf{a} * E_B$ due to the commutativity of the group elements. The work remained theoretical due to the lack of efficient subroutines.

In fact, the expensive computational cost has been the longstanding major bottleneck for the real-world application of the isogeny-based NIKE. For example, the subsequent refinements, notably by De Feo, Kieffer, and Smith [DKS18], improved the runtime. However, the performance is still far from efficient, taking hundreds of seconds on a normal PC. Meanwhile, Castryck, Lange, Martindale, Panny, and Renes [CLM⁺18] re-engineer the CRS paradigm in a supersingular setting over prime fields, and give the first efficient post-quantum NIKE called CSIDH, based on a commutative isogeny class-group action. From then on, a series of works aim to achieve a faster and constant-time implementation of CSIDH [MR18, MCR19, OAYT19, CCC⁺19, HLKA20, CDRH22, NOTT23]. However, the same commutativity that enables the NIKE functionality also exposes CSIDH to subexponential-time quantum attacks via hidden-shift algorithms [Kup05, Kup13]. As a result, CSIDH's concrete quantum security has been debated and repeatedly re-estimated [BLMP19, BS20, CSCJR22]. The analyses push CSIDH to parameter sizes approximately eight times larger than originally envisioned. For example, to achieve NIST-I security level, the underlying prime is required to be of 2048 (aggressive) or 4096 (conservative) bits in length to have sufficiently large discriminant. These larger parameters lead to the somewhat impractical performance. The state-of-art implementation credits to [CCSC⁺24, BHS25] which implement various constant-time CSIDH variants. Under the conservative choice targeting NIST-I level, the optimized C/assembly implementation of CSIDH-4096 and CTIDH-4096 still require tens of gigacycles per group action. Practically, when embedded into real protocols, this translates into handshake latencies on the order of a few to tens of seconds for OPTLS-style early-authentication handshakes under the parameters, which makes CSIDH-based NIKes difficult to deploy outside niche, high-latency/low-bandwidth environments [CCSC⁺24].

Recently, Houben [Hou26] proposed a new formulation to instantiate the ideal class group actions based on supersingular elliptic curves. The curves are equipped with orientations determined by imaginary quadratic orders of a large discriminant. In contrast to CSIDH, this framework allows one to scale the discriminant, and hence the group size, without increasing the size of the base field. By combining with the constant-time technique [Hou25], the resulting class group action admits an evaluation algorithm that is fully deterministic, strictly constant time and dummy free, and can be implemented without conditional branches [Hou26]. While evaluating such non-Frobenius orientations is more complex (notably requiring arithmetic over $\mathbb{F}_{p^2}$ rather than $\mathbb{F}_p$), the ability to work with much smaller primes provides a substantial performance lever [Hou26]. It naturally implies a non-interactive key exchange protocol, referred to as OSIDH-LD in this paper. In particular, the reported proof-of-concept SageMath implementation targeting NIST-I security level over a 256-bit prime runs in a median 53.4 seconds per action on a single core of a PC with a 3.30GHz CPU, which is more than 7 times faster than the unoptimized SageMath implementation of dCSIDH-4096. These results indicate promising performance potential for this direction, and it would be both interesting and of great significance to

conduct further optimizations on OSIDH-LD.

Moreover, the novel isogeny group action is well-suited for parallel implementations. In essence, the procedure can be decomposed into several subroutines, which operate in an identical manner but with distinct inputs. To the best of our knowledge, there is currently no optimization approach that takes into account enhancing the parallel performance of OSIDH-LD in the literature.

1.1 Contribution

Based on the above analysis, this paper aims to enhance the practicality of OSIDH-LD. The contributions of this paper are twofold. First, we optimize OSIDH-LD by presenting algorithmic optimizations, and provide a C implementation to validate the correctness of these optimizations and demonstrate the performance improvements. Second, to further enhance the performance, we batch the independent subroutines in the group action and develop two parallel implementations of OSIDH-LD, by leveraging multiple threads and AVX-512 vector extensions, respectively. The multi-threaded implementation can take advantage of multi-core processors, while the AVX-512 implementation can accelerate data processing through vectorization. To the best of our knowledge, this work is the first to implement OSIDH-LD in C and AVX-512, which not only enriches the implementation methods of OSIDH-LD but also provides a reference for future high-performance design. The following elaborates on our main contributions:

- **Theoretical Optimizations:**

- **Key Agreement Improvement.** In key agreement, two parties share the j -invariant of an elliptic curve as the secret. However, the group action proceeds by updating the entire information of the orientation at each loop, which requires large computational overhead. Based on this observation, we propose a *tail pruning* technique for key agreement to avoid redundant calculations, with a $1.24\times$ speedup.
- **Isomorphism Generation Optimization.** In OSIDH-LD, both parties compute their public keys and the shared secret by performing the class group actions multiple times. At the end of each loop during the class group action, one should compute an isomorphism to identify the codomain. Instead of using the current expensive computation method via invariant differentials, we consider how to employ the isomorphism computation algorithm used in the SQIsign implementation [AAA⁺24] to generate such an isomorphism, leading to a $1.17\times$ acceleration.
- **Isogeny Computation Strategy Acceleration.** This is a well-known technique used in isogeny-based protocols. Targeting the ideal class group action in OSIDH-LD, we propose a specific cost estimation method for the optimal strategy under the fixed prime factor ordering, and find an effective strategy for isogeny computations. Consequently, the new strategy achieves an approximately 20% reduction in the overall computational cost.

- **Parallel Implementation:**

- OSIDH-LD exhibits inherently parallelizable characteristics, making it exceptionally amenable to parallel acceleration techniques. We develop parallel implementations of OSIDH-LD, including thread-based parallelism and vectorization-based parallelism. Notably, we achieve near perfect parallelism in key generation, and consider how to batch the computational tasks when using tail pruning technique in key agreement. Our implementations are engineered for low latency rather than high throughput, i.e., our technique optimizes computing a single

key generation rather than doing multiple key generations at the same time. In particular, our AVX-512IFMA implementation offers a $4.97\times$ speedup in key generation of OSIDH-LD, which is close to optimal.

1.2 Related work

The most well-known isogeny-based (interactive) key exchange is SIDH [JD11]. SIDH is significantly more efficient than the CRS scheme. Its KEM version, the supersingular isogeny key encapsulation SIKE [JAC⁺22], was once regarded as a strong contender in the NIST post-quantum cryptography standardization process. Unfortunately, due to a series of effective attacks [CD23, MMP⁺23, Rob23], both SIDH and SIKE have been completely broken. Since then, several SIDH-like schemes have been proposed [BF23, BMP23] to resist these attacks, but they are less efficient than SIDH and require much larger public keys due to the larger characteristic of the underlying base field. The recent advancements [NO24, BM25, CCLL26] give the performances comparable to SIKE while resisting to the attacks. In contrast, CSIDH [CLM⁺18] and its derived schemes [DG19, LGD21, BDK⁺22] remain unaffected by the attacks.

The inefficiency nature of isogeny-based protocols motivates research into AVX-512 acceleration techniques. Orisaka, Aranha, and López [OLHA18] utilized AVX-512 vector instructions to accelerate the finite field modular multiplication in SIDH, while Kostic and Gueron [KG19] optimized SIKE by using VPMADD effectively. Subsequently, Cheng et al. [CFGR22] employed AVX-512IFMA to accelerate both base field arithmetic and isogeny computations in SIKE. However, the characteristic prime differs fundamentally from the prime structure in OSIDH-LD, which restricts direct applicability of these optimizations.

In [CFG⁺21], Cheng et al. conducted an in-depth investigation into leveraging AVX-512 vector constructions to accelerate CSIDH. They focused on vectorized finite-field arithmetic and batching multiple instances, improving throughput while maintaining constant-time considerations. The OSIDH-LD class group action shares some similarities with CSIDH. For example, they both compute isogenies whose degrees are multiples of small primes. However, as we will illustrate in Section 4, OSIDH-LD is well-suited for parallel implementations without batching multiple key generation/agreement instances. Instead, we study how to batch the subroutines within one instance, which is more flexible and attractive in applications.

Roadmap. The rest of this paper is organized as follows. Section 2 provides the necessary backgrounds. We propose several optimizations to improve OSIDH-LD in Section 3 and introduce our batching approach in Section 4. We report the experimental results in Section 5. Finally, we draw our concluding remarks and future work in Section 6.

2 Preliminary

This section recalls the basic knowledge required in this paper. We also briefly review the deterministic class-group action of OSIDH-LD. For more details, we refer the reader to [Sil09, Voi21, Hou26].

Notation. Define $[m_1, m_2]$ to be the set of integers s such that $m_1 \leq s \leq m_2$, and $[m_1, m_2]^n$ as the set of vectors $(s_1, s_2, \dots, s_n)$ where $m_1 \leq s_i \leq m_2$. For a vector $(s_1, s_2, \dots, s_n)$, we write $(s_i)_{i=1}^n$ for simplicity. Vector addition is component-wise: $(a_i)_{i=1}^n + (b_i)_{i=1}^n := (a_i + b_i)_{i=1}^n$. We call $(a_i)_{i=1}^n$ a multiple of $(b_i)_{i=1}^n$ if $a_i = mb_i$ for all i with some $m \in \mathbb{Z}$.

2.1 Elliptic curves, isogenies and endomorphisms

Elliptic curves. Define E to be an elliptic curve over $\mathbb{F}_{p^2}$ of characteristic $p > 3$. The set of $\mathbb{F}_{p^2}$ -rational points of E is defined as

$$E(\mathbb{F}_{p^2}) = \{(x, y) \in (\mathbb{F}_{p^2})^2 : y^2 = x^3 + Ax + B\} \cup \{0_E\},$$

where 0_E is the *point at infinity*. In particular, $\#E(\mathbb{F}_{p^2}) = p^2 + 1 - t$, where $|t| \leq 2p$. A curve E is called *supersingular*, if $p \mid t$.

Isogenies. An *isogeny* φ is a morphism between two elliptic curves that maps the point at infinity to the point at infinity. An isogeny φ is *separable* if $\#\ker(\varphi) = \deg(\varphi) = \ell$, abbreviated as an ℓ -isogeny. The *dual* isogeny is denoted by $\hat{\varphi}$, satisfying that $\hat{\varphi} \circ \varphi = [\ell]$. If $\ker(\varphi)$ is a cyclic group, then we call the isogeny φ *cyclic*. Further, if φ is an ℓ -isogeny with kernel $\langle P \rangle$ for some $P \in E[\ell]$, then $\ker(\hat{\varphi}) = \langle \varphi(Q) \rangle$, where Q is a rational point such that $\langle P, Q \rangle = E[\ell]$. Given a point $K \in E[\ell]$ of order ℓ coprime to p , one can compute an ℓ -isogeny $\varphi : E \rightarrow E'$ such that $E[\varphi] := \ker(\varphi) = \langle K \rangle$ by Vélú's formula [Vél71, BdFLS20]. Isogeny computations are the vital operations in isogeny-based cryptography. To compute an isogeny by Vélú's formula efficiently, the degree is required to be smooth, i.e., it is a multiple of small primes.

The following algorithms are the basic components in isogeny computations:

- $\text{Mul}(E, P, N)$: Given a point P defined on E and a positive integer N , outputs $[N]P$.
- $\text{Isog}(E, \mathcal{L}, K, N)$: Given a point K of order N defined on E and a list of points $\mathcal{L} = \{P_1, P_2, \dots, P_n\}$, outputs the codomain E' of the isogeny $\varphi : E \rightarrow E'$ with kernel $\langle K \rangle$, and the evaluations of φ at P_i , $i = 1, 2, \dots, n$.

Endomorphisms. An *endomorphism* is an isogeny from a curve E to itself. All endomorphisms of E forms a ring $\text{End}(E)$ under addition and composition. The invertible elements of $\text{End}(E)$ form the automorphism group of E , denoted by $\text{Aut}(E)$. When E has j -invariant 0 or 1728, there exists non-trivial automorphisms. Otherwise, $\text{Aut}(E) = \{\pm 1\}$. If two curves have the same j -invariant, then they are isomorphic, and vice versa.

2.2 Orientations

Let $K = \mathbb{Z}[\sigma]$ be an imaginary quadratic number field. A K -*orientation* is an injective ring homomorphism $\iota : K \rightarrow \text{End}^0(E) := \text{End}(E) \otimes_{\mathbb{Z}} \mathbb{Q}$. We call (E, ι) a K -oriented elliptic curve. Given an imaginary quadratic order $\mathcal{O} \subset K$, we call ι an $\mathcal{O}$ -*orientation* if $\iota(\mathcal{O}) \subset \text{End}(E)$. An $\mathcal{O}$ -orientation is *primitive* if it does not extend to a strictly larger imaginary quadratic order. For $\alpha \in \text{End}^0(E)$, let $\text{tr}(\alpha)$ and $\text{Nrd}(\alpha)$ denote the reduced trace and reduced norm, and $\text{Disc}(\alpha) := \text{tr}(\alpha)^2 - 4\text{Nrd}(\alpha)$.

Induced orientations. Let $\varphi : E \rightarrow E'$ be a separable isogeny of degree $\deg(\varphi)$. Given an orientation ι on E , the induced orientation $\varphi_*(\iota)$ on E' is defined by

$$\varphi_*(\iota)(\alpha) := \varphi \circ \iota(\alpha) \circ \hat{\varphi} \otimes \deg(\varphi)^{-1} \in \text{End}^0(E').$$

Two $\mathcal{O}$ -oriented curves (E, ι) and (E', ι') are K -oriented isomorphic if there exists an isomorphism $\psi : E \rightarrow E'$ such that $\iota' = \psi_*(\iota)$.

2.3 Class group actions

Let $\mathcal{O} \subset K$ be an imaginary quadratic order. Let $I(\mathcal{O})$ denote the group of invertible fractional $\mathcal{O}$ -ideals, and $P(\mathcal{O})$ the subgroup of principal fractional ideals. The ideal class group is $\text{Cl}(\mathcal{O}) := I(\mathcal{O})/P(\mathcal{O})$.

For an invertible ideal $\mathfrak{a} \subset \mathcal{O}$ of norm coprime to p and an $\mathcal{O}$ -oriented curve (E, ι) over $\mathbb{F}_{p^2}$, the Deuring correspondence attaches an isogeny $\varphi_{\mathfrak{a}} : E \rightarrow E'$ with kernel

$$E[\mathfrak{a}] := \bigcap_{\alpha \in \mathfrak{a}} E[\alpha],$$

and we write $E' = [\mathfrak{a}]E$ (well-defined up to isomorphism), or equivalently $\mathfrak{a} * E$ when emphasizing the action. If $\mathfrak{a} \sim \mathfrak{b}$, i.e., they are in the same class in $\text{Cl}(\mathcal{O})$, then

$$([\mathfrak{a}]E, (\varphi_{\mathfrak{a}})_*(\iota)) \cong ([\mathfrak{b}]E, (\varphi_{\mathfrak{b}})_*(\iota))$$

as K -oriented elliptic curves. Hence, $\text{Cl}(\mathcal{O})$ acts on $\mathcal{O}$ -oriented curves up to K -oriented isomorphism; when restricted to *primitive* $\mathcal{O}$ -orientations, the action is free. Beyond the NIKE, when the group structure is known, an *effective* group action [ADMP20] enables various cryptographic primitives [BKV19, ADMP20, DM20, KLLQ23, HLM⁺25], which is not within the scope of this work.

The following proposition, roughly speaking, allows us to track kernel generators easily along a chain of group actions.

Proposition 1 ([Hou26, Proposition 2.1]). *Let $\mathcal{O} = \mathbb{Z}[\sigma]$ be an imaginary quadratic order and let (E, ι) be a primitively $\mathcal{O}$ -oriented elliptic curve over a field k . Assume that $\text{Nrd}(\sigma)$ is coprime to both $\text{char}(k)$ and $\text{Disc}(\mathcal{O})$ (equivalently, to both $\text{char}(k)$ and $\text{tr}(\sigma)$). Then*

- (a) $E[\iota(\sigma)] \cap E[\iota(\bar{\sigma})] = \{0_E\}$.
- (b) *If $m \in \mathbb{Z}_{>0}$ satisfies $m \mid \text{Nrd}(\sigma)$, then the $\mathcal{O}$ -ideal $\mathfrak{a} := (m, \sigma)$ is invertible. Moreover, if $\varphi_{\mathfrak{a}} : E \rightarrow \mathfrak{a} * E$ denotes the corresponding isogeny and ι' is the induced orientation on $\mathfrak{a} * E$, then $(\mathfrak{a} * E)[\iota'(\bar{\sigma})] = \varphi_{\mathfrak{a}}(E[\iota(\bar{\sigma})])$.*

2.4 Kernel representations

In the group action setting as mentioned above, the orientation is an essential element representing a set member. The orientations are typically represented by using the (full) kernel representation.

Kernel representation. Fix a smooth integer $M = \prod_{i=1}^n \ell_i^{e_i}$ with small primes ℓ_i , and suppose $E[M] \subseteq E(\mathbb{F}_{p^2})$. A *kernel representation* of an endomorphism σ of degree M^r is a chain of cyclic isogenies

$$E = E_1 \xrightarrow{\varphi_1} E_2 \xrightarrow{\varphi_2} \dots \xrightarrow{\varphi_r} E_{r+1} = E$$

where $\deg(\varphi_j) = M$ and $\ker(\varphi_j) = \langle P_j \rangle$ for some $P_j \in E_j(\mathbb{F}_{p^2})$ of order M , $j = 1, 2, \dots, r$.

Full kernel representation. Let $Q_{j+1} \in E_{j+1}(\mathbb{F}_{p^2})$ generate $\ker(\widehat{\varphi_j})$. Then the tuple

$$\mathcal{E} := \{(E_j, P_j, Q_j) \mid 1 \leq j \leq r\}$$

is called a *full kernel representation* of the induced $\mathbb{Z}[\sigma]$ -orientation. In practice, a crucial property is

$$\langle P_j \rangle \cap \langle Q_j \rangle = \{0_{E_j}\} \quad \text{for all } j,$$

which (in the orientation framework) is equivalent to the relevant small primes not ramifying in the class group. It holds in particular in common parameter regimes where $\text{Nrd}(\sigma)$ and $\text{tr}(\sigma)$ are coprime (cf. [Hou26]).

2.5 OSIDH-LD

Now we briefly introduce the novel class group action formulation introduced by Houben in [Hou26], and the derived key exchange protocol, referred to as OSIDH-LD in this paper.

Let p to be a prime such that $p^2 - 1$ has big enough smooth divisor, and assume that $M \mid p^2 - 1$ with $M = \ell_1^{e_1} \ell_2^{e_2} \cdots \ell_n^{e_n}$, where ℓ_i , $i = 1, 2, \dots, n$ are small primes. Let $\mathcal{O} \subset K = \mathbb{Z}[\sigma]$, where σ has reduced norm M^r . At a high level, given a full kernel representation of the $\mathcal{O}$ -orientation $\mathcal{E}$ and a vector $(s_i)_{i=1}^n \in [0, e_i]_{i=1}^n$, one can compute each tuple of $[\prod_{i=1}^n \ell_i^{s_i}] * \mathcal{E} := \{(E'_j, P'_j, Q'_j)\}_{j=1}^r$ as follows:

1. Constructs isogenies $\varphi_{j,1}^+$ with kernel $\langle [M/\ell_1^{s_1}](P_j) \rangle$, and isogenies $\varphi_{j,1}^-$ with kernel $\langle [M/\ell_1^{s_1}](Q_{j+1}) \rangle$;¹
2. For i ranging from 2 to n , constructs the isogenies $\varphi_{j,i}^+$ with kernel $\langle [M/\ell_i^{s_i}] \varphi_{j,i-1}^+ \circ \cdots \circ \varphi_{j,1}^+(P_j) \rangle$, and $\varphi_{j,i}^-$ with kernel $\langle [M/\ell_i^{e_i-s_i}] \varphi_{j,i-1}^- \circ \cdots \circ \varphi_{j,1}^-(Q_{j+1}) \rangle$;
3. Let $\varphi_j^+ := \varphi_{j,n}^+ \circ \cdots \circ \varphi_{j,1}^+$, $\varphi_j^- := \varphi_{j,n}^- \circ \cdots \circ \varphi_{j,1}^-$. The curve E' is set to be the codomain of φ_j^+ (φ_j^-), and points $P'_j = \varphi_j^+(P_{j+1})$, $Q'_j = \varphi_j^-(Q_j)$.

Algorithm 1 summarizes the main procedure, which is fully deterministic, strictly constant-time and dummy-free. Figure 1 illustrates the deterministic group action when $r = 4$. Here $\text{cswap}(A, B, b)$ denotes a constant-time conditional swap: it swaps the values of the variables A and B iff $b = 1$.

Algorithm 1: Class group action evaluations on full kernel representations

Input: A full kernel representation $\mathcal{E} = \{(E_j, P_j, Q_j) \mid 1 \leq j \leq r\}$ of $\mathbb{Z}[\sigma]$ -orientation on E , a vector $(s_1, s_2, \dots, s_n) \in [0, e_i]_{i=1}^n$.
Output: A full kernel representation $\mathcal{E}' = \{(E'_j, P'_j, Q'_j) \mid 1 \leq j \leq r\}$ of $\mathbb{Z}[\sigma]$ -orientation on $E' = [\prod_{i=1}^n \ell_i^{s_i}]E$.

```

1 for  $j = 1, \dots, r$  do
2    $R_0 \leftarrow (E_j, P_j, Q_j)$ ,  $R_1 \leftarrow (E_{j+1}, Q_{j+1}, P_{j+1})$ ,  $m \leftarrow \prod_{i=1}^n \ell_i^{e_i}$ ;
3   for  $i = 1, \dots, n$  do
4     for  $t = 1, \dots, e_i$  do
5        $b \leftarrow t > s_i$ ;
6        $\text{cswap}(R_0, R_1, b)$ ;
7        $m \leftarrow m/\ell_i$ ,  $K \leftarrow \text{Mul}(R_0[0], R_0[1], m)$ ;
8        $R_0[0], \{R_0[1], R_0[2]\} \leftarrow \text{Isog}(R_0[0], \{R_0[1], R_0[2]\}, K, \ell_i)$ ;
9        $R_1[1] \leftarrow \text{Mul}(R_1[0], R_1[1], \ell_i)$ ;
10       $\text{cswap}(R_0, R_1, b)$ ;
11    end
12  end
13   $(R_0[1], R'_j) \leftarrow (R_1[2], R_0)$ ;
14 end
15 return  $R'_1, \dots, R'_r$ .
```

Remark 1. [Isomorphism Test] It is easy to see that the codomains of φ_j^+ and φ_j^- are isomorphic. However, in computations it may happen that the codomains of φ_j^+ and φ_j^- ($j = 1, 2, \dots, r$) do not have the same coefficients, differed by an isomorphism. In this case, an isomorphism is required to identify the codomain. In the implementation, one can choose a full kernel representation of the orientation such that only E'_r needs to be identified. Currently, the method for isomorphism computation is expensive [Hou26]. In Section 3.2 we will propose an efficient approach to optimize the performance.

¹ We use the notational convention that $Q_1 = Q_{r+1}$.

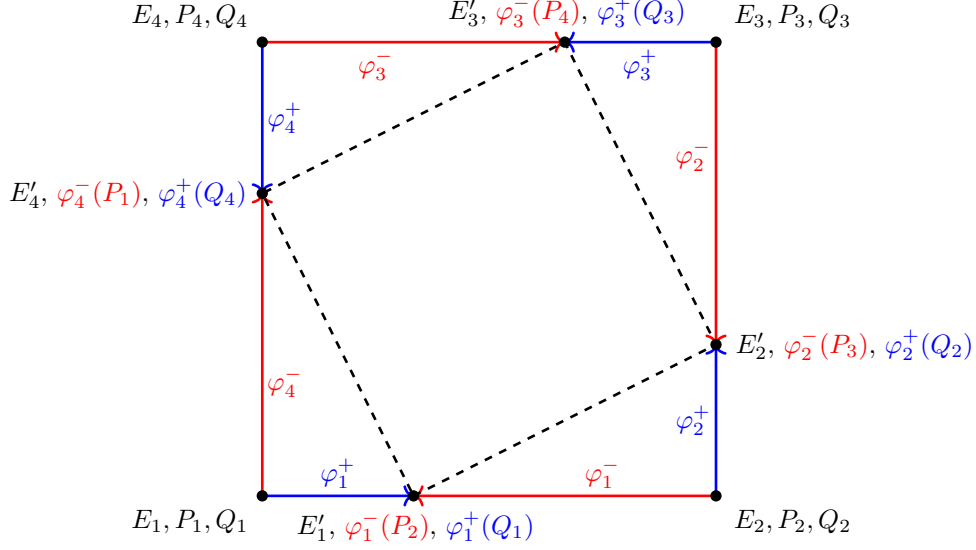


Figure 1: A sketch of deterministic class group action.

Based on the above class group action, it is easy to derive the following non interactive key exchange protocol.

Setup. The setup algorithm of OSIDH-LD on input the security parameter λ returns the public parameter $\mathbf{pp} = (p, M, r, \iota, \sigma, E_0, B)$. Here, $p \in \mathbb{N}$, $M \in \mathbb{N}$, are stated as above and (E_0, ι) is an $\mathcal{O}$ -oriented elliptic curve, where $E_0 : y^2 = x^3 + x$ is a supersingular curve defined over $\mathbb{F}_{p^2}$ with j -invariant 1728, ι is represented by the full kernel representation, and $\mathcal{O} = \mathbb{Z}[\sigma]$ satisfying $\text{tr}(\sigma) < 4(\text{Nrd}(\sigma) - \sqrt{\text{Nrd}(\sigma)})$ and $\text{Nrd}(\sigma) = M^r$. Let $\mathcal{E}_0$ be a full kernel representation of the $\mathcal{O}$ -orientation. Furthermore, $B \in \mathbb{N}$ is the bound such that the size of the private key space

$$\#\{(s_i)_{i=1}^n \mid s_i \in [0, Be_i]\} = \prod_{i=1}^n (1 + Be_i) \geq 2^{2\lambda}.$$

Key Generation. Alice first selects her secret key $\mathbf{sk}_A = (a_i)_{i=1}^n$ where $a_i \in [0, Be_i]$. After separating the secret key into $\{\mathbf{sk}_{A,i}\}_{i=1}^B$ such that $\mathbf{sk}_{A,i} \in [0, e_i]^n$ and $\sum_{i=1}^B \mathbf{sk}_{A,i} = \mathbf{sk}_A$, she computes $\mathcal{E}_A = \prod_{i=1}^n \mathfrak{l}_i^{a_i} * \mathcal{E}_0$ using B applications of Algorithm 1. Her public key is $\mathcal{E}_A$. Similar to Alice, Bob chooses a random vector $\mathbf{sk}_B = (b_i)_{i=1}^n$ as his secret key and computes $\mathcal{E}_B = \prod_{i=1}^n \mathfrak{l}_i^{b_i} * \mathcal{E}_0$, and finally sends $\mathcal{E}_B$ to Alice.

Key Agreement. After receiving $\mathcal{E}_B$, Alice computes $\mathcal{E}_{AB} = \prod_{i=1}^n \mathfrak{l}_i^{a_i} * \mathcal{E}_B$ with respect to her secret key. Analogously, Bob computes $\mathcal{E}_{BA} = \prod_{i=1}^n \mathfrak{l}_i^{b_i} * \mathcal{E}_A$. The shared secret is the j -invariant of the first curve of the kernel representation.

Define

$$\bar{\omega} : \mathbb{Z}^n \rightarrow \text{Cl}(\mathcal{O}), \quad (s_i)_{i=1}^n \rightarrow \prod_{i=1}^n \mathfrak{l}_i^{s_i}. \quad (1)$$

Similar to CSIDH [CLM⁺18, Section 7.1], under the Cohen-Lenstra heuristics [CL84] and the Gaussian heuristic, the group homomorphism $\bar{\omega}$ maps the secret key space $[0, Be_i]_{i=1}^n$ approximately injectively into the class group. From Proposition 2, it is provably injective except for the trivial collision $(0)_{i=1}^n \equiv (e_i)_{i=1}^n$ when $B = 1$.

Proposition 2 ([Hou25, Proposition 3.1]). *Suppose that $\text{tr}(\sigma)^2 < 4(\text{Nrd}(\sigma) - \sqrt{\text{Nrd}(\sigma)})$. Let $0 \leq s_i \leq e_i$ be integers. If $(s_1, \dots, s_n) \in \ker(\bar{\omega})$, then either $s_i = 0$ for all i , or $s_i = e_i$ for all i .*

2.6 AVX-512

Single Instruction Multiple Data (SIMD) is a high-throughput computing approach that enables data-parallel processing by executing the same operation concurrently across multiple data sets. This paradigm is exemplified by AVX-512 (Advanced Vector Extensions 512), which offers 32 512-bit wide general-purpose registers, significantly accelerating compute-intensive tasks such as cryptographic operations on x86 CPUs.

AVX-512IFMA (AVX-512 Integer Fused Multiply-Add) is an attractive functional extension of AVX-512. It provides instructions for fused multiply-and-accumulate operations for 52-bit wide integers. This special feature makes it an ideal candidate for expediting specific high-precision integer arithmetic operations. In this work, we adapt AVX-512IFMA to accelerate the performance of OSIDH-LD.

Regarding hardware support, AVX-512IFMA has not yet achieved full adoption in the consumer desktop processors. AMD has provided complete 512-bit data path support for AVX-512 since the Zen 5 architecture, including the IFMA extensions. On the other hand, Intel abandoned AVX-512 support on desktop processors after the 11th generation Core series, limiting it to server/workstation-grade processors only. However, this situation is expected to change with the launch of the Nova Lake series in late 2026.² According to Intel, the Nova Lake processors will support the AVX10.2 extension[Int25], which will include support for AVX-512 instructions such as IFMA[Int26].

3 Algorithmic Optimizations

In this section, we present a series of techniques aimed at enhancing the performance of OSIDH-LD. It is important to note that the techniques described here are not confined to parallel implementations alone. Therefore, they are applicable across a wide range of implementation types.

3.1 Key Agreement Improvement

We propose a *tail pruning* technique that accelerates the *key-agreement evaluation* rather than the core class-group action algorithm itself. In the key agreement phase of OSIDH-LD, following the standard NIKE recipe from commutative group actions, each party evaluates the class-group action (Algorithm 1) for B iterations and both parties ultimately output the shared secret as the j -invariant of the final curve E_{AB} .

Our key observation is that, to obtain the final shared j -invariant, the "full" computation of Algorithm 1 is not always necessary. Concretely, following the standard recipe, the implementation shall evaluate the class group action by updating the complete endomorphism information, which enables for the "next" class group action to construct the entire endomorphism data, whereas only the j -invariant of the final curve E_{AB} is needed. That is, the current specification contains redundant computations, which could take up a large portion in the computation up to the public parameters B, r .

The improvement is given in Algorithm 2. The detailed reasoning is as follows. Recall that in Section 2.5 and Algorithm 2: the computation for each tuple $\{E'_j, P'_j, Q'_j\}$ in $\mathcal{E}'$ can be determined solely by two tuples $\{E_j, P_j, Q_j\}$ and $\{E_{j+1}, P_{j+1}, Q_{j+1}\}$ in $\mathcal{E}$. Therefore, in the last iteration, it suffices to update only the first two tuples $(\{E_1, P_1, Q_1\}, \{E_2, P_2, Q_2\})$, obtained from the penultimate iteration, to obtain the final curve E_{AB} ; to this end, in

² <https://www.pcmag.com/news/intel-confirms-nova-lake-core-ultra-400-cpus-launching-this-year>

the penultimate iteration, it suffices to update only the first three tuples; and so on. By recursion, at the $(B - r)$ -th iteration, it is necessary to have the entire information of the full kernel representation of the orientation to compute E_{AB} . In other words, the $(B - r)$ -th iteration is exactly the iteration where we can start our pruning process.

Algorithm 2: Class group action evaluations at the b iteration in key agreement
($b > B - r$)

Input: A full kernel representation $\{(E_j, P_j, Q_j) \mid 1 \leq j \leq r\}$ of $\mathbb{Z}[\sigma]$ -orientation on E , a vector $(s_1, s_2, \dots, s_n) \in [0, e_i]^n$, and the execution iteration $b \in [B + 1 - r, B]$.

Output: $E' = [\prod_{i=1}^n \ell_i^{s_i}]E$.

```

1 for  $j = 1, \dots, B + 1 - b$  do
2    $R_0 \leftarrow (E_j, P_j, Q_j)$ ,  $R_1 \leftarrow (E_{j+1}, Q_{j+1}, P_{j+1})$ ,  $m \leftarrow \prod_{i=1}^n \ell_i^{e_i}$ ;
3   for  $i = 1, \dots, n$  do
4     for  $t = 1, \dots, e_i$  do
5        $b' \leftarrow t > s_i$ ;
6        $\text{cswap}(R_0, R_1, b')$ ;
7        $m \leftarrow m / \ell_i$ ,  $K \leftarrow \text{Mul}(R_0[0], R_0[1], m)$ ;
8        $R_0[0], \{R_0[1], R_0[2]\} \leftarrow \text{Isog}(R_0[0], \{R_0[1], R_0[2]\}, K, \ell_i)$ ;
9        $R_1[1] \leftarrow \text{Mul}(R_1[0], R_1[1], \ell_i)$ ;
10       $\text{cswap}(R_0, R_1, b')$ ;
11    end
12  end
13   $R_0[1] \leftarrow R_1[2]$ ,  $R'_j \leftarrow R_0$ ;
14 end
15 return  $R_1[0]$ .
```

By integrating the pruned process into the NIKE application, Alice and Bob compute their shared secret as follows: in the first $B - r$ iterations the algorithm they execute Algorithm 1 as previous; then from the b -th ($b > B - r$) iteration they iterate j from 1 to $B + 1 - b$ rather than to r .

The savings are immediate. For an integer $b > B - r$, at the b -th iteration, the update of the last $(b + r - B)$ -th tuples will be skipped. Consequently, we eliminate a total of $\sum_{t=1}^r (t - 1) = r(r - 1)/2$ tuple updates in the key agreement phase. Compared to the standard evaluation with r tuple updates, for the parameter set $(B, r) = (35, 13)$ from the implementation from [Hou26], we observe a theoretic $\approx 1.24\times$ speedup of the key agreement. Notably, the pruning process depends only on public iteration indices, and thus preserves the deterministic, dummy-free, and constant-time feature of [Hou26]. Finally, we remark that this technique is not specific to NIKE only: it is possible to apply to other applications of restricted effective group actions to be instantiated by [Hou26] when only the resulting j -invariant is required in some subroutines (e.g., [DG19, BKP20, BDK⁺22]).

3.2 Isomorphism Generation Optimization

As mentioned in Remark 1, one shall use an isomorphism to identify E'_r . To tackle this problem, the current SageMath implementation [Hou26] computes an action on invariant differentials, which is currently quite costly. To improve this subroutine, we adapt the algorithm based on [CS18], which is also applied in the SQIsign implementation [AAA⁺24, Section 2.2.1.1] to construct the isomorphism. Although it involves one square root operation and one inversion over $\mathbb{F}_{p^2}$, it is still more efficient than action on invariant differentials according to our experiments.

A subtle but crucial challenge in our implementation arises from the impact of isomorphism operations on the linear independence of P_1 and Q_1 , i.e.,

$$E_1 / \langle [M/\ell_j]P_i \rangle \cong E_1 / \langle [M/\ell_j]Q_i \rangle \text{ for some } j. \quad (2)$$

Especially, it only happens when the curve E_1 has non-trivial automorphisms, i.e., the j -invariant of E_1 is 0 or 1728.

A natural approach to address this problem is to generate an orientation that does not involve the curves with j -invariant 0 or 1728. Moreover, it should be guaranteed that during the class group action evaluation, the orientation also excludes the curves with j -invariant 0 or 1728. In the following, we provide a practical approach to avoid handling the issue that the full kernel representation involves the curves with j -invariant 0 or 1728 with overwhelming probability. Before that, we introduce Proposition 3 to show the special property of the orientation with small-trace endomorphism.

Proposition 3. *Assume that $\mathcal{E} = \{(E_j, P_j, Q_j) \mid 1 \leq j \leq r\}$ is a full kernel representation of the $\mathcal{O}$ -orientation with $\mathcal{O} \subset K = \mathbb{Z}[\sigma]$, where σ satisfies that $\text{tr}(\sigma)^2 < 4(\text{Nrd}(\sigma) - \sqrt{\text{Nrd}(\sigma)})$. Let $\mathcal{E}' = \{(E'_j, P'_j, Q'_j) \mid 1 \leq j \leq r\}$ be the output of Algorithm 1 with input $\mathcal{E} = \{(E_j, P_j, Q_j) \mid 1 \leq j \leq r\}$ and $(s_i)_{i=1}^n \in [0, re_i]_{i=1}^n$ ³. If the curve E_1 is involved in the full kernel representation $\mathcal{E}'$, then $(s_i)_{i=1}^n$ is a multiple of $(e_i)_{i=1}^n$.*

Proof. We first prove that the endomorphism σ is not a composition of two endomorphisms of E_1 . We proceed similarly as the proof of Proposition 2 [Hou25, Proposition 3.1]. Assume that $\tau \in \mathbb{Z}[\sigma]$, which is an endomorphism decomposed from σ . Then we can suppose that $(\tau) = \prod_{i=1}^n \mathfrak{l}_i^{s_i}$. Since σ is primitive, $\tau \notin \mathbb{Z}$. By possibly replacing $(s_i)_{i=1}^n$ and $(e_i - s_i)_{i=1}^n$, assume that $\text{Nrd}(\tau) \leq \sqrt{\text{Nrd}(\sigma)}$. However,

$$4\sqrt{\text{Nrd}(\sigma)} < 4\text{Nrd}(\sigma) - \text{tr}(\sigma)^2 = |\text{Disc}(\sigma)| \leq |\text{Disc}(\tau)| \leq 4\text{Nrd}(\tau) \leq 4\sqrt{\text{Nrd}(\sigma)},$$

which is a contradiction.

Let $\mathcal{S} = \{j \in [1, r] \mid E'_j = E_1\}$. Then from the above deduction we know that $\#\mathcal{S} \leq 1$. If $E'_1 = E_1$, then from Proposition 2 we know that either $s_i = re_i$ for all i or $s_i = 0$ for all i . If $E'_j = E_1$ for some $j = 2, \dots, n$, then from $\#\mathcal{S} \leq 1$ we know that there exists at most one corresponding vector such that $E'_j = E_1$. In particular, it is easy to verify that

$$\left[\prod_{i=1}^n \mathfrak{l}_i^{(r-j)e_i} \right] E_j = \left[\prod_{i=1}^n \mathfrak{l}_i^{(r-j-1)e_i} \right] E_{j+1} = \dots = \left[\prod_{i=1}^n \mathfrak{l}_i^{e_i} \right] E_r = E_1,$$

i.e., $((r-j)e_i)_{i=1}^n$ is the corresponding vector such that $E'_j = E_1$, which is a multiple of $(e_i)_{i=1}^n$. This completes the proof. $\square$

From Proposition 3, it is straightforward to deduce that under the OSIDH-LD setting, the full kernel representation $[\mathfrak{a}]\mathcal{E}_0$ will contain E_0 if $(s_i)_{i=1}^n$ is a multiple of $(e_i)_{i=1}^n$. To avoid this issue, we modify the definition of $\mathcal{E}_0$: we perform class group actions to compute $[\mathfrak{a}]\mathcal{E}_0$ where $\mathfrak{a} = \overline{\omega}(s_0)$ with $s_0 = (2Be_1 + 1, 0, \dots, 0)$, and replace $\mathcal{E}_0$ with $[\mathfrak{a}]\mathcal{E}_0$ as the starting set element in the public parameter. Since the vector $s_0 + s$ is not a multiple of $(e_i)_{i=1}^n$ for all $s \in [0, 2Be_i]^n$, we avoid the aforementioned problem, except when $[\overline{\omega}(s_0)]E_0 = [\overline{\omega}(s)]E_0$. Similar to CSIDH [CLM⁺18, Section 7.1], OSIDH-LD is based on the heuristic assumption that $\overline{\omega}$ is a roughly injective map from the key space to the class groups [Hou26, Section 5.2]. Hence, under the heuristic assumption, the probability that $[\overline{\omega}(s_0)]E_0 = [\overline{\omega}(s)]E_0$ is negligible and thus the curves with j -invariant 0 or 1728 are not involved in the full kernel representation of $\mathcal{O}$ -orientation during the class group actions in key generation and key agreement of OSIDH-LD with an overwhelming probability.

³ Note that the vector is not involved in $[0, e_i]^n$. However, we can execute Algorithm 1 r times by separating $(s_i)_{i=1}^n$ into r vectors that are elements in $[0, e_i]^n$.

3.3 Isogeny Computation Strategy Acceleration

Clearly, Algorithm 1 employs a multiplicative-based strategy to compute isogenies. The complexity is $O(n^2)$, which is far from optimal. A well-established improvement is to adopt a balanced strategy. This can effectively reduce the complexity to $O(n \log n)$. The core idea of this balanced strategy is to iteratively split a strategy into two sub-strategies, making the number of leaves in the two sub-strategies as equal as possible. Figure 4 gives toy examples of the two strategies.

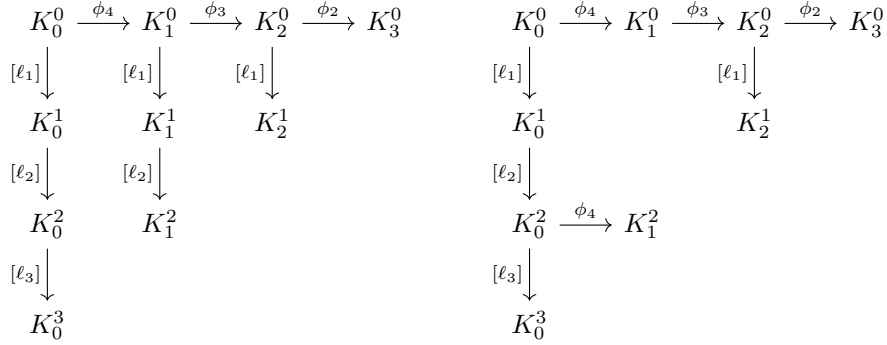


Figure 4: Toy examples for isogeny computations $\phi = \phi_1 \circ \phi_2 \circ \phi_3 \circ \phi_4$ with kernel $\langle K \rangle$, where $\deg(\phi_i) = \ell_i$, $i = 1, 2, 3, 4$ and $K_{j_2}^{j_1} = [\prod_{j=0}^{j_1} \ell_j] \phi_5 \circ \dots \circ \phi_{5-j_2}(K)$, where $\ell_{j_0} = 1$ and $\phi_5 = \text{id}$. Note that the points that have the same subscript index are defined on the same supersingular curve. The left one is the multiplicative-based strategy, while the right one is the balanced strategy.

Indeed, it is possible to find a more effective strategy. In the SIDH framework, the isogeny to be computed has a degree that is a prime power $\ell^\bullet$. In such a scenario, an optimal strategy, which is defined as the strategy with the lowest cost among all strategies having the same leaves, can be identified in polynomial time [FJP14]. Define $C(k)$ to be the cost of an optimal strategy with k leaves. Then the optimal strategy can be determined with respect to the following equation:

$$C(k) = \begin{cases} 0, & \text{if } k = 1, \\ \min \{C(i) + C(k-i) + c_v + c_h \mid 0 \leq i \leq k-1\}, & \text{if } k > 1, \end{cases}$$

where c_v, c_h are the costs from the root of the overall strategy to the roots of the two sub-strategies, respectively. When we assign the weights $q_1, q_2 > 0$ to the vertical edges (corresponding to the cost of scalar multiplication by ℓ) and the horizontal edges (corresponding to the cost of ℓ -isogeny evaluation), respectively, then $c_v = (k-i)q_1$ and $c_h = iq_2$.

However, in CSIDH and OSIDH-LD, one has to compute an isogeny whose degree is a multiple of various small primes. As stated in [CDRH22, Section 3.4.1], it is infeasible to find an optimal strategy and ordering of the small prime factors. Drawing on the insights from [CDRH22], we can expect that the optimal prime factor ordering strategy will lead to a computational cost that closely approximating that of processing isogenies in ascending degree. Under the fixed prime factor ordering, the authors of [CDRH22] can find a strategy for CSIDH which is nearly optimal.

Here we focus our attention to finding an effective strategy that is close to optimal for OSIDH-LD. Heuristically, we also arrange the prime factors from smallest to largest. At a cursory glance, the process of devising a strategy for OSIDH-LD might seem identical to that of CSIDH. In what follows, we will present a novel approach for determining the optimal strategy for OSIDH-LD, based on the following observation.

To be more precise, let R_0 and R_1 denote the tuples in Algorithm 1. When traversing the vertical path from the root of the main strategy to that of the sub-strategy, typically two scalar multiplications need to be executed. The reason lies in the fact that during the execution of the sub-strategy, the information of both R_0 and R_1 might be required simultaneously. However, if the sub-strategy is a leaf node, only one of R_0 and R_1 is necessary to generate a small-prime degree isogeny. In this situation, one can determine the point by the secret key and then just perform one scalar multiplication.

As a result, we have the following equation, which can be used to determine the optimal strategy under the fixed prime factor ordering $L = \{\ell_1, \ell_2, \dots, \ell_k\}$:

$$C_L(k) = \begin{cases} 0, & \text{if } k = 1, \\ \min \begin{cases} C_{L_1}(1) + C_{L_2}(k-1) + c_h + c_v, \\ C_{L_1}(i) + C_{L_2}(k-i) + c_v + c_h + c', \end{cases} & \text{if } k > 1, \end{cases} \quad (3)$$

where $L_1 = \{\ell_1, \ell_2, \dots, \ell_i\}$, $L_2 = \{\ell_{i+1}, \ell_{i+2}, \dots, \ell_k\}$ and c' is the cost of the scalar multiplication by $\prod_{i=1}^k \ell_i$.⁴

Consequently, we have devised an effective strategy for isogeny computations. As indicated in [CDRH22], the obtained strategy for CSIDH is also inclined towards a multiplicative-based approach. This preference can be attributed to the superior performance of scalar multiplication by ℓ over the ℓ -isogeny evaluation when ℓ is large. To elaborate, scalar multiplication by ℓ can be executed with polynomial complexity in $\log(\ell)$. In contrast, ℓ -isogeny evaluation has a complexity of $O(\ell)$ (or $\tilde{O}(\sqrt{\ell})$). This significant difference in computational complexity makes scalar multiplication a more attractive option. Moreover, it is evident that Equation (3) also leans towards suggesting a multiplicative-based strategy. When $i = 1$ and $k > 1$, a substantial amount of scalar multiplication with a cost of c' can be saved. This cost-saving aspect further reinforces the tendency of the obtained strategy to be multiplicative-based.

4 Parallel Optimizations

There are primarily two approaches for parallelly accelerating the performance of the OSIDH-LD protocol. The first one is to batch key generation/agreement instances. Unlike CSIDH [CFG⁺21], the class group action has the deterministic property. Therefore, the batching computations can be done efficiently with low latency. The second approach is to consider how to parallelize the procedures in a single key generation/agreement. The latter approach imposes stricter algorithmic constraints, particularly regarding data independence. However, it is more attractive because it offers greater flexibility in applications.

In OSIDH-LD, Alice and Bob need to execute B applications of Algorithm 1 to compute their public keys and shared secret. Each update of the full kernel representation of the orientation requires the complete information from the previous one. As a result, the data dependency prevents them from parallelizing these B executions.

Nonetheless, Algorithm 1 is highly amenable to parallelization. The computation of (E'_j, P'_j, Q'_j) in the class group action depends solely on (E_j, P_j, Q_j) and $(E_{j+1}, P_{j+1}, Q_{j+1})$, rather than the full kernel representation of the orientation: the curve E'_j is the codomain of φ_j^+ and φ_j^- , points $P'_j = \varphi_j^-(P_{j+1})$ and $Q'_j = \varphi_j^+(Q_j)$. Therefore, the computation of the r tuples in $\mathcal{E}'$ can be significantly accelerated through parallel processing. We also note that the operations performed in each iteration are identical under the constant-time algorithm, making it particularly suitable for optimization with SIMD instructions. In this section, we will describe our parallel implementations of OSIDH-LD, including thread-based parallelism and vectorized parallelism based on AVX-512IFMA instructions.

⁴ It should be noted that c_v and c_h are also dependent on the prime factor ordering L . For simplicity, we do not emphasize the dependence or define additional notations. For in-depth details, we refer to [CDRH22].

4.1 Multi-threaded implementation

In our multi-threaded implementation, we adopt the fork-join parallel execution model, which means executing n tasks in parallel, and then executing another n tasks in parallel after all of them are completed. When the number of threads $n = r$, Algorithm 1 perfectly evaluates a group action in parallel. But in our setting, there are at most 8 accessible threads, while $r = 13$. Fortunately, we find that Algorithm 1 does not need to complete one group action before starting the next one.

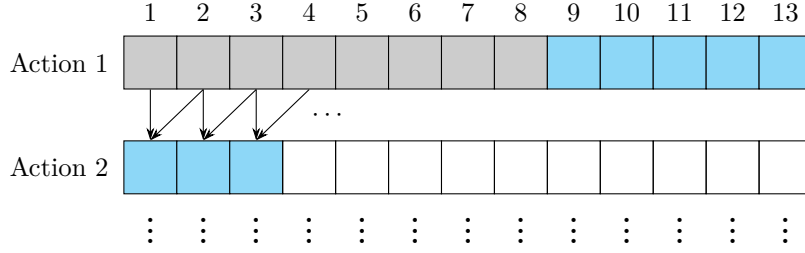


Figure 5: A sketch of multi-threaded key generation implementation. We set $n = 8$ and $r = 13$.

In Figure 5, each row represents a group action, and each grid represents the M -isogeny that needs to be computed between two elliptic curves, with gray indicating completed ones, blue indicating ongoing ones, and white indicating pending ones. The computations of the last three ongoing threads rely on the outputs of the first four threads in the previously completed execution, and hence the ongoing execution proceeds successfully. Indeed, it is easy to see that in our structure the information required for the next parallel execution has already been computed during the current or the previous parallel execution of eight isogenies. In particular, the information used for the first k ($k < n$) M -isogeny computations in the next action has been obtained, so these k isogenies can be computed together with the remaining $n - k$ isogenies in the previous action. This means that even though the number of threads $n < r$, we can still achieve near perfect parallelism in the key generation phase, i.e., every operation runs with n -way parallelism except the very last parallel step.

The situation for key agreement becomes more complex when using the technique proposed in Section 3.1. The number of isogenies required for each group action gradually decreases in the later portion of the computation. As soon as an action needs no more M -isogenies than the number of threads n , the remaining work no longer maintains perfect n -way parallelism. As Figure 6 shows, for instance, the task marked with red crosses depends on two tuples, in which one of them is not yet available. Therefore, it leaves only seven tasks ready to run. Although it is not a perfect parallelism, employing the computational saving technique in Section 3.1 still leads to a better performance.

We also attempted to develop an alternative thread-scheduling strategy: a task queue backed by a thread pool. An example of this strategy is shown in Figure 7. Since different threads may finish their isogeny computations at slightly different moments, a thread that completes its current assignment does not need to wait for the others at a join barrier. Instead, it can immediately pick any other ready task, as sketched in Figure 7. In theory, this yields higher performance by reclaiming the cycles that would otherwise be lost to synchronization. However, our experiments showed that, apart from greatly complicating the code, the approach delivered no measurable gain—most likely owing to the heavy locking it requires. A deeper investigation of its tuning and trade-offs is left as future work.

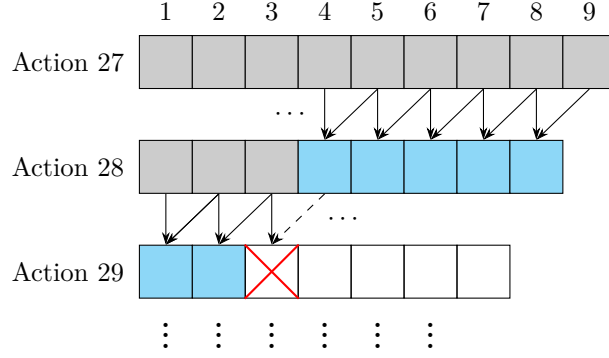


Figure 6: A sketch of multi-threaded key agreement implementation. Under the parameter setting which targets the NIST-I security level, the parameter $B = 35$ and $r = 13$, and we set $n = 8$. The task marked with red crosses cannot be batched in the ongoing execution because one of the required tuples is not available.

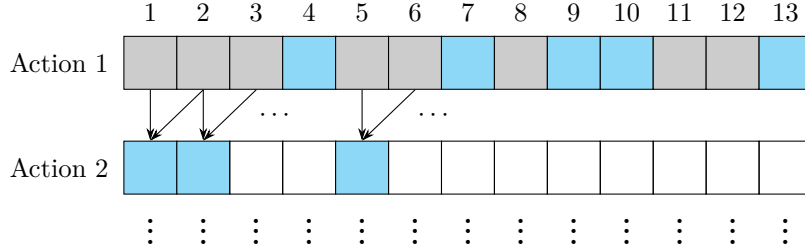


Figure 7: An example of thread-scheduling strategy.

4.2 AVX-512IFMA implementation

Our finite-field data structures and algorithms are similar to the AVX-512IFMA implementation of CSIDH [CFG⁺21]. Given that our parameter set would result in an extra limb with radix- 2^{51} , we opt to use radix- 2^{52} instead. Consequently, an $\mathbb{F}_p$ element is represented as $f = f_0 + 2^{52}f_1 + 2^{104}f_2 + 2^{156}f_3 + 2^{208}f_4$, where $0 \leq f_j < 2^{52}$, $j = 0, 1, \dots, 4$. This representation allows field elements to be up to 260-bit during computations. Given radix- 2^{52} representations of eight elements $a, b, c, d, e, f, g, h \in \mathbb{F}_p$, we define the following limb vector set $\mathbf{V}$:

$$\mathbf{V} = \langle a, b, c, d, e, f, g, h \rangle = \left\{ \begin{bmatrix} a_0, b_0, c_0, d_0, e_0, f_0, g_0, h_0 \\ a_1, b_1, c_1, d_1, e_1, f_1, g_1, h_1 \\ a_2, b_2, c_2, d_2, e_2, f_2, g_2, h_2 \\ a_3, b_3, c_3, d_3, e_3, f_3, g_3, h_3 \\ a_4, b_4, c_4, d_4, e_4, f_4, g_4, h_4 \end{bmatrix} \right\} = (V_0, V_1, V_2, V_3, V_4),$$

where each $V_i = [a_i, b_i, c_i, d_i, e_i, f_i, g_i, h_i]$ is called a *limb vector*. Performing addition, subtraction, or multiplication on two limb vectors is similar to conducting these operations on two integers modulo p . In short, the AVX-512IFMA implementation enables us to execute eight identical $\mathbb{F}_p$ (or $\mathbb{F}_{p^2}$) operations at the same time.

At the algorithmic level, the AVX-512IFMA implementation bears a strong resemblance to an 8-thread parallel implementation. In fact, it can be viewed as an eight-way parallel computation that incurs no fork-join overhead. The key difference lies in the requirement that every elementary operation must be strictly lane-aligned. This strict alignment condition poses a potential challenge. During the implementation, each ℓ -isogeny may

proceed in a different direction depending on the secret key. However, this issue can be effectively resolved by a correct constant-time implementation of the `cswap` routine in Algorithm 1. By utilizing bitwise masks, we selectively swap only the targeted vector lanes while keeping the remaining lanes unchanged. As a result, a single vectorized invocation can simultaneously handle isogenies with different directions.

Same as the multi-threaded implementation, the vectorized implementation also suffers from reduced parallelism toward the end of the key agreement phase. This effect is particularly pronounced given that our current design relies on an 8-way vectorized $\mathbb{F}_{p^2}$ pipeline. Incorporating 4-way and 2-way AVX-512IFMA routines for $\mathbb{F}_{p^2}$, together with the corresponding elliptic-curve operations could marginally enhance the speed of key agreement generation. We note this as a direction for future research.

5 Implementation

In Section 5.1, we employ C programming language integrated with assembly language for field arithmetic, which is adopted to demonstrate the practical impacts of our proposed methods. Subsequently, we present the experimental results of both the multi-threaded implementation and the AVX512-IFMA implementation in Section 5.2.

Our implementations⁵ are executed on an Intel Core i7-11700F CPU with a base frequency of 2.50 GHz. To ensure the stability and reproducibility of the test environment, both hyper-threading and turbo boost technologies are not available. The operating system is Ubuntu 24.04 LTS, and the code is compiled with Clang 18.1.3 at the -O3 optimization level with link-time optimization enabled.

5.1 C Implementation

To the best of our knowledge, the only publicly available implementation of OSIDH-LD is provided by Houben⁶, which is coded in SageMath. To conduct a fair efficiency comparison, we take Houben’s code as a foundation and reimplement OSIDH-LD in C, incorporating assembly language for field arithmetic. For elliptic-curve arithmetic and isogeny computation we reuse the C code in the first-round NIST submission of the SQIsign implementation⁷. The finite-field layer is produced with the constant-time assembly generator shipped with CTIDH⁸, lightly edited to fit our parameters.

For parameters, we apply the setup given in Houben’s code, which slightly differs from the example in the paper [Hou26]. In the following, we list the field characteristic p , the non-twisted isogeny degree M^+ and the twisted isogeny degree M^- (note that $M = M^+M^-$).

$$\begin{aligned} p &= 0x53d1c0debd0c1ba3edd760148d21a078008f642a9f26f65995c4d86fc6efff7 \\ M^+ &= 0x24bed52b01b8d020e25aecd3b301b2fbf1d75d56d89d48d7b \\ &= 3^4 \cdot 5 \cdot 7 \cdot 11^2 \cdot 19 \cdot 23 \cdot 29 \cdot 31 \cdot 41 \cdot 43 \cdot 47 \cdot 53 \cdot 71 \cdot 79 \cdot 83 \cdot 89 \cdot 97 \cdot 107 \\ &\quad \cdot 109 \cdot 113 \cdot 131 \cdot 137 \cdot 149 \cdot 151 \cdot 157 \cdot 163 \cdot 167 \cdot 181 \cdot 229 \cdot 239 \cdot 269 \\ M^- &= 0x3acebf3dd1f3042710750fce762eef \\ &= 13 \cdot 17 \cdot 37 \cdot 59 \cdot 61 \cdot 67 \cdot 73 \cdot 101 \cdot 103 \cdot 127 \cdot 139 \cdot 173 \cdot 179 \cdot 191 \cdot 193 \\ &\quad \cdot 199 \cdot 211 \cdot 241 \end{aligned}$$

To further improve the performance, we consider two alternative data structures for the full kernel representation of an orientation, and examine their impact on performance. To store such a representation, we can choose one of two layouts: AoS (Array of Structures) or SoA (Structure of Arrays).

⁵ github.com/wwwweize/OSIDH-LD ⁶ github.com/houbenmr/LargeDiscriminantOrientations

⁷ github.com/SQIsign/the-sqisign/tree/nist-v1 ⁸ ctidh.isogeny.org/high-ctidh-20210523.tar.gz

AoS layout: This layout stores an array where each element is a complete record. The AoS layout of the full kernel representation has the form:

$$\mathcal{E} := \{(E_j, P_j, Q_j) \mid 1 \leq j \leq r\},$$

which is the intuitive layout already mentioned earlier.

SoA layout: This layout keeps a separate array for every data field. The SoA layout is of the form:

$$\mathcal{E} := (\{E_j \mid 1 \leq j \leq r\}, \{P_j \mid 1 \leq j \leq r\}, \{Q_j \mid 1 \leq j \leq r\})$$

The SoA paradigm offers a multitude of possible configurations, and the example presented above is merely one specific instantiation. In our implementation, we delve deeper into the design of the SoA structure. For instance, when dealing with the elliptic curve $E_j : y^2 = x^3 + A_j x^2 + x$, where $A_j = a_j/c_j$ is in projective form, we use two separate arrays for data storage: one array is dedicated to storing the values of $a_j + 2c_j$, while the other stores $4c_j$. The reason why we store $a_j + 2c_j$ and $4c_j$ instead of directly storing a_j and c_j stems from the efficiency of the isogeny computations. Consequently, each of these arrays is populated with $\mathbb{F}_{p^2}$ elements.

Table 1 summarizes the performance numbers of our implementations. All the implementations are equipped with differential addition chains for scalar multiplications. The first column indicates the data layout employed, and the second column shows which isomorphism generation method is used (ID for invariant differentials, IF for isomorphism formulae). The third column records whether the strategy-based isogeny acceleration is available. The Gcyc and Time columns report the experimental results (expressed in gigacycles and seconds, respectively). We take the key generation results with AoS layout, ID and without strategy as the baseline to compute the speedup ratio.

Table 1: Benchmark of OSIDH-LD in different optimization.

AoS/SoA	Isom.	Stra.	KeyGen			ImKeyAgree		
			Gcyc	Time	Speedup	Gcyc	Time	Speedup
AoS	ID	no	19.08	7.645	1.00×	15.41	6.143	1.24×
SoA	ID	no	18.17	7.278	1.05×	14.64	5.865	1.30×
AoS	ID	yes	15.56	6.234	1.23×	12.41	4.973	1.54×
SoA	ID	yes	15.23	6.103	1.25×	12.14	4.865	1.57×
AoS	IF	no	16.30	6.529	1.17×	13.49	5.406	1.41×
SoA	IF	no	15.48	6.202	1.23×	12.84	5.142	1.49×
AoS	IF	yes	12.80	5.127	1.49×	10.57	4.236	1.80×
SoA	IF	yes	12.25	4.907	1.56×	10.19	4.084	1.87×

All else being equal, implementing OSIDH-LD with the SoA layout will be marginally faster than with the AoS layout. This might imply that SoA should always be chosen for peak-speed implementation of OSIDH-LD. However, several factors influence the performance outcome, including cache capacity, alignment policy, and the design of the underlying finite-field arithmetic. These factors can significantly affect the performance. Therefore, determining the fastest layout requires specific tuning when implementing OSIDH-LD in other platforms.

Furthermore, Table 1 demonstrates that all the techniques proposed in Section 3 significantly enhance the performance of the OSIDH-LD implementation. Specifically, when integrating all the improvements, we achieve a 1.56×

5.2 Parallel Implementation

For the ease of data reading/writing in the parallel implementations, we adapt the AoS layout instead of the SoA layout. All the theoretic techniques in Section 3 are also applied. The multi-threaded implementation is built on OpenMP (version 18.1.3), and the timings are reported in Figure 8. Since precise CPU-cycle counts are impractical under multi-threading, we only report wall-clock times. The two curves in Figure 8 show the theoretical run-times for key generation and key agreement, obtained by dividing the single-thread times in Row 7 of Table 1 by the number of threads, respectively. The scattered points are the measured values.

We highlight that due to cache contention, shared-memory effects, and scheduler-induced variability, the OpenMP-based implementation is not constant-time and may leak side-channel information. However, as shown in Figure 8, the close match confirms that the algorithm parallelizes almost perfectly. The key agreement results deviate slightly more than the key generation ones, and the gap widens as the thread count increases, which are consistent with our analysis in Section 4.1. Experimental results also indicate that the techniques in Section 3 are well-suited for parallel implementations. This provides important insights for optimizing parallel implementations of OSIDH-LD on other platforms.

#threads	2	3	4	5	6	7	8
KeyGen (s)	2.557	1.707	1.282	1.025	0.856	0.733	0.674
ImKeyAgree (s)	2.138	1.438	1.089	0.888	0.749	0.648	0.609

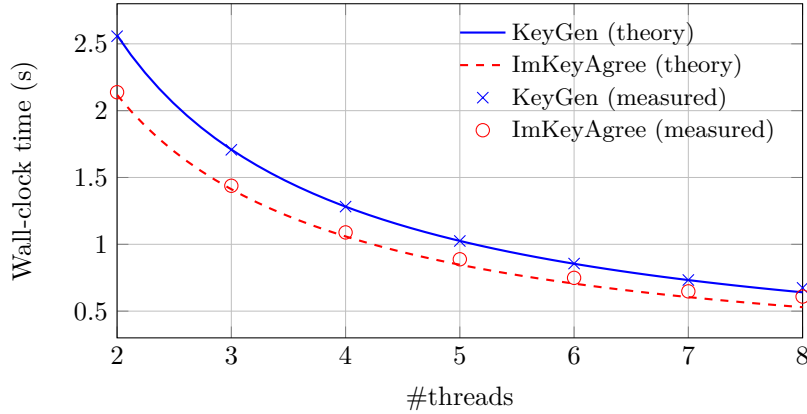


Figure 8: Wall-clock time of multi-threaded OSIDH-LD.

Table 2 reports the comparisons between the C implementation with assembly language for field arithmetic and our AVX-512IFMA implementation. The results show that the AVX-512IFMA implementation achieves an acceleration factor of 4.97 in key generation and 4.62 in key agreement. In the following, we show that our throughput-optimized AVX-512 implementation is close to the theoretical optimum.

For simplicity, we take the multiplication operation as an example, which is also the main operation in the implementation. Note that we use five 52-bit limbs to represent an $\mathbb{F}_p$ element. As we mentioned in Section 2.6, an AVX-512IFMA instruction is able to work on 52-bit elements. To perform an $\mathbb{F}_p$ multiplication, we need 25 limb operations. On the other hand, the x64 implementation requires $(256/64)^2 = 16$ multiplication instructions for one $\mathbb{F}_p$ multiplication. Therefore, the optimal acceleration factor is $16/25 \times 8 = 5.12$ in theory. This is consistent with our experimental results of key generation, which is a near-perfect parallelism, achieving a $4.97\times$ speedup. The efficiency gain of improved key agreement is slightly lower. This is because by using the tail pruning technique in

Section 3.1, the implementation is not a perfect parallelism in the last several iterations, as we discussed in Section 4.1.

Table 2: Benchmark of OSIDH-LD implemented by AVX-512IFMA.

Implementation	KeyGen			ImKeyAgree		
	Gcyc	Time	Speedup	Gcyc	Time	Speedup
C + Assembly	12.80	5.127	-	10.57	4.236	-
AVX-512IFMA	2.574	1.031	4.97×	2.289	0.917	4.62×

5.3 Comparison

In Table 3, we compare our optimized implementation of OSIDH-LD with state-of-the-art constant-time isogeny-based NIKE schemes (without key validation) instantiated from commutative group actions [CCSC⁺24, CHMR25, BHS25]. Among existing implementations only [CCSC⁺24, BHS25] achieve deterministic and dummy-free simultaneously, and only [CCSC⁺24] achieve a conservative NIST-I target [CSCJR22]. By combining the OSIDH-LD’s new group-action formulation with our optimizations, we obtain an improvement of more than a 4× factor over this baseline [CCSC⁺24]. Compared with [BHS25], our key generation implementation remains comparable while offering better security strength. We remark that due to the distinct group action formulations of CSIDH and OSIDH-LD, the techniques do not seem to be easily transferable in between. Overall, these results indicate that OSIDH-LD is a promising direction for practical post-quantum NIKE.

Table 3: Comparison with state-of-the-art constant-time isogeny-based NIKE in giga cycles. We run the benchmark together with dCSIDH and dCTIDH from the codes available in [CCSC⁺24, CHMR25], respectively on the platform mentioned above. Here DT. and DF. denote deterministic and dummy-free implementations resp. The security strength is estimated by matching the discriminant size to the CSIDH-2048 and CSIDH-4096, corresponding to the aggressive and conservative NIST-I resp [CSCJR22].

Scheme	KeyGen	KeyAgree	DT.DF.		CSIDH Strength
dCSIDH2048[CCSC ⁺ 24]	12.43	7.927	✓	✓	2048
dCSIDH4096[CCSC ⁺ 24]	67.92	40.97	✓	✓	4096
dCTIDH2048-205[CHMR25]	12.39	2.620	✓	✗	2048
dCTIDH2048-194[CHMR25]	13.70	2.516	✓	✗	2048
Hardened dCTIDH2048-205[BHS25]	10.69	2.688	✓	✓	2048
Hardened dCTIDH2048-194[BHS25]	9.085	2.598	✓	✓	2048
This work	12.25	10.19	✓	✓	4096

6 Conclusion and Future Work

6.1 Conclusion

This paper presents optimizations of OSIDH-LD at both the theoretical and practical levels. Firstly, we introduce several algorithmic techniques to improve OSIDH-LD, including key agreement computation savings, isomorphism generation improvements, and isogeny computation strategy acceleration. Notably, these techniques are independent and can be applied to optimize OSIDH-LD. Secondly, we present a parallelism method tailored for OSIDH-LD, and leverage AVX-512 vector extensions to enhance the performance. To our knowledge, this is the first work to present low-level OSIDH-LD implementations, and take parallelism into consideration. Although OSIDH-LD remains less efficient than other

post-quantum key exchange protocols (such as lattice-based schemes), we believe that it has the potential to become more competitive with continued research.

6.2 Future Work

We discuss a potential future work in this subsection. Further optimizations for the parallel implementations of OSIDH-LD are left for future work. Besides, as mentioned in Section 3.2, our approach to avoid handling the curves with j -invariant 0 or 1728 during the class group action is heuristic. It is interesting to provide a normal proof without heuristic assumption, or propose a more stable solution. Lastly, we introduce a potential approach to further improve OSIDH-LD, although currently it does not work in our implementation.

Recall that in OSIDH-LD, both parties should perform Algorithm 1 by B times to obtain the public key in key generation, or the shared secret in key agreement. The size of B determines the performance of OSIDH-LD. It is natural to ask how to make B smaller. Here we propose a potential approach to reduce B . The core idea lies in the specific structure of the full kernel representation of the $\mathcal{O}$ -orientation. In the current protocol, both parties share the j -invariant of $E_{AB} \cong E_{BA}$ of the first tuple as the secret. However, there are r elliptic curves involved within the full kernel representation of the $\mathcal{O}$ -orientation. Therefore, an optimization strategy is to utilize the order of the tuples in the full kernel representation of the $\mathcal{O}$ -orientation, to modify the private key space, leading to a smaller B . Except for the vector $s \in [0, Be_i]_{i=1}^n$, one further select an integer $m \in [0, r-1]$, and set (s, m) as the secret key. The private key space is

$$\mathcal{K} := \{(s, m) \mid s \in [0, Be_i]_{i=1}^n, m \in [0, r-1]\}.$$

Correspondingly, the protocol proceeds as follows:

Key Generation. Alice first selects her secret key $\mathbf{sk}_A = (r_A, (a_i)_{i=1}^n) \in [0, r-1] \times [0, Be_i]^n$. After computing $\mathcal{E}'_A = \prod_{i=1}^n \mathfrak{l}_i^{a_i} * \mathcal{E}_0 = \{(E_{A,i}, P_{A,i}, Q_{A,i}) \mid 1 \leq i \leq r\}$ using B applications of Algorithm 1, she states the public key is the tuple $\mathcal{E}_A = \{(E_{A,r_A}, P_{A,r_A}, Q_{A,r_A}), \dots, (E_{A,r}, P_{A,r}, Q_{A,r}), (E_{A,1}, P_{A,1}, Q_{A,1}), \dots, (E_{A,r_A-1}, P_{A,r_A-1}, Q_{A,r_A-1})\}$. Similar to Alice, Bob chooses a random vector $\mathbf{sk}_B = (r_B, (b_i)_{i=1}^n) \in [0, Be_i]^n$ as his secret key and computes $\mathcal{E}_B = \prod_{i=1}^n \mathfrak{l}_i^{b_i} * \mathcal{E}_0 = \{(E_{B,i}, P_{B,i}, Q_{B,i}) \mid 1 \leq i \leq r\}$, and sends $\mathcal{E}_B = \{(E_{B,r_B}, P_{B,r_B}, Q_{B,r_B}), \dots, (E_{B,r}, P_{B,r}, Q_{B,r}), (E_{B,1}, P_{B,1}, Q_{B,1}), \dots, (E_{B,r_B-1}, P_{B,r_B-1}, Q_{B,r_B-1})\}$ to Alice.

Key Agreement. After receiving $\mathcal{E}_B$, Alice computes $\mathcal{E}_{AB} = \prod_{i=1}^n \mathfrak{l}_i^{a_i} * \mathcal{E}_B$ with respect to her secret key. Analogously, Bob computes $\mathcal{E}_{BA} = \prod_{i=1}^n \mathfrak{l}_i^{b_i} * \mathcal{E}_A$. The shared secret is the j -invariant of the r_A -th (r_B -th) curve of $\mathcal{E}_{AB}$ ($\mathcal{E}_{BA}$).

Note that the private key modification results in extra collisions. For example, in the case that $s \in [0, Be_i - 1]_{i=1}^n$, $s + (1)_{i=1}^n \in [0, Be_i]$, and thus we have collisions $(0, s) \equiv (r-1, s + (1)_{i=1}^n)$ and $(j, s) \equiv (j-1, s + (1)_{i=1}^n)$ for $j \in [1, r-1]$. Under the heuristic assumption that $\bar{\omega}$ is roughly injective, the extra class groups corresponds to the vectors $(s_i + m)_{i=1}^n$, where $s_i + m > Be_i$ for some i , and $m \in [0, r-1]$. As a result, the extra private key space is

$$\mathcal{K}_{\text{ex}} = \{((s_i)_{i=1}^n, m) \in [0, Be_i]_{i=1}^n \times [0, r-1] \mid s_i + m > Be_i \text{ for some } i\},$$

In this way we can reduce the extra private key space from B to $B-1$. However, this technique is not directly comparable to the current implementation. This is because the isomorphism, which is used to identify E'_r , exposes the value of m . How to apply this technique into the implementation, or find out a better approach to make B smaller based on our observation, are left for future work.

Acknowledgement

Weize Wang, Kaizhan Lin and Yunlei Zhao are supported by the National Key Research and Development Program of China (No.2022YFB2701601), Shanghai Collaborative Innovation Fund (No.XTCX-KJ-2023-54), Special Fund for Key Technologies in Blockchain of Shanghai Scientific and Technological Committee (No.23511100300), and National Natural Science Foundation of China (No. 12441107). Yi-Fu Lai was funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under Germany's Excellence Strategy - EXC 2092 CASA - 390781972.

References

- [AAA⁺24] Marius A. Aardal, Gora Adj, Diego F. Aranha, Andrea Basso, Isaac Andrés Canales Martínez, Jorge Chávez-Saab, Maria Corte-Real Santos, Pierrick Dartois, Luca De Feo, Max Duparc, Jonathan Komada Eriksen, Tako Boris Fouotsa, Décio Luiz Gazzoni Filho, Basil Hess, David Kohel, Antonin Leroux, Patrick Longa, Luciano Maino, Michael Meyer, Kohei Nakagawa, Hiroshi Onuki, Lorenz Panny, Sikhar Patranabis, Christophe Petit, Giacomo Pope, Krijn Reijnders, Damien Robert, Francisco Rodríguez Henríquez, Sina Schaeffler, and Benjamin Wesolowski. SQIsign. Technical report, National Institute of Standards and Technology, 2024. available at <https://csrc.nist.gov/Projects/pqc-dig-sig/round-2-additional-signatures>.
- [ADMP20] Navid Alamati, Luca De Feo, Hart Montgomery, and Sikhar Patranabis. Cryptographic group actions and applications. In Shiho Moriai and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2020, Part II*, volume 12492 of *Lecture Notes in Computer Science*, pages 411–439, Daejeon, South Korea, December 7–11, 2020. Springer, Cham, Switzerland.
- [BdFLS20] Daniel J. Bernstein, Luca de Feo, Antonin Leroux, and Benjamin Smith. Faster computation of isogenies of large prime degree. In Steven Galbraith, editor, *ANTS-XIV - 14th Algorithmic Number Theory Symposium*, volume 4 of *Proceedings of the Fourteenth Algorithmic Number Theory Symposium (ANTS-XIV)*, pages 39–55, Auckland, New Zealand, June 2020. Mathematical Sciences Publishers.
- [BDK⁺22] Ward Beullens, Samuel Dobson, Shuichi Katsumata, Yi-Fu Lai, and Federico Pintore. Group signatures and more from isogenies and lattices: Generic, simple, and efficient. In Orr Dunkelman and Stefan Dziembowski, editors, *Advances in Cryptology – EUROCRYPT 2022, Part II*, volume 13276 of *Lecture Notes in Computer Science*, pages 95–126, Trondheim, Norway, May 30 – June 3, 2022. Springer, Cham, Switzerland.
- [BF23] Andrea Basso and Tako Boris Fouotsa. New SIDH Countermeasures for a More Efficient Key Exchange. In Jian Guo and Ron Steinfeld, editors, *ASIACRYPT 2023, Part VIII*, volume 14445 of *LNCS*, pages 208–233. Springer, Singapore, December 2023.
- [BHS25] Gustavo Banegas, Andreas Hellenbrand, and Matheus Saldanha. Hardened CTIDH: Dummy-free and deterministic CTIDH. *LNCS*, pages 194–215, 2025.
- [BKP20] Ward Beullens, Shuichi Katsumata, and Federico Pintore. Calamari and Falafi: Logarithmic (Linkable) Ring Signatures from Isogenies and Lattices. In Shiho Moriai and Huaxiong Wang, editors, *ASIACRYPT 2020, Part II*, volume 12492 of *LNCS*, pages 464–492. Springer, Cham, December 2020.

- [BKV19] Ward Beullens, Thorsten Kleinjung, and Frederik Vercauteren. CSI-FiSh: Efficient isogeny based signatures through class group computations. In Steven D. Galbraith and Shiho Moriai, editors, *Advances in Cryptology – ASIACRYPT 2019, Part I*, volume 11921 of *Lecture Notes in Computer Science*, pages 227–247, Kobe, Japan, December 8–12, 2019. Springer, Cham, Switzerland.
- [BLMP19] Daniel J. Bernstein, Tanja Lange, Chloe Martindale, and Lorenz Panny. Quantum Circuits for the CSIDH: Optimizing Quantum Evaluation of Isogenies. In Yuval Ishai and Vincent Rijmen, editors, *EUROCRYPT 2019, Part II*, volume 11477 of *LNCS*, pages 409–441. Springer, Cham, May 2019.
- [BM25] Andrea Basso and Luciano Maino. POKÉ: A compact and efficient PKE from higher-dimensional isogenies. In *EUROCRYPT 2025, Part II*, LNCS, pages 94–123. Springer, Cham, June 2025.
- [BMP23] Andrea Basso, Luciano Maino, and Giacomo Pope. FESTA: Fast encryption from supersingular torsion attacks. In Jian Guo and Ron Steinfeld, editors, *Advances in Cryptology – ASIACRYPT 2023, Part VII*, volume 14444 of *Lecture Notes in Computer Science*, pages 98–126, Guangzhou, China, December 4–8, 2023. Springer, Singapore, Singapore.
- [BS20] Xavier Bonnetain and André Schrottenloher. Quantum Security Analysis of CSIDH. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part II*, volume 12106 of *LNCS*, pages 493–522. Springer, Cham, May 2020.
- [CCC⁺19] Daniel Cervantes-Vázquez, Mathilde Chenu, Jesús-Javier Chi-Domínguez, Luca De Feo, Francisco Rodríguez-Henríquez, and Benjamin Smith. Stronger and Faster Side-Channel Protections for CSIDH. In Peter Schwabe and Nicolas Thériault, editors, *LATINCRYPT 2019*, volume 11774 of *LNCS*, pages 173–193. Springer, Cham, October 2019.
- [CCLL26] Shiping Cai, Mingjie Chen, Yi-Fu Lai, and Kaizhan Lin. PIKE: Faster isogeny-based public key encryption with pairing-assisted decryption. In *Public-Key Cryptography – PKC 2026*, 2026. To appear. Cryptology ePrint Archive, Paper 2026/473.
- [CCSC⁺24] Fabio Campos, Jorge Chávez-Saab, Jesús-Javier Chi-Domínguez, Michael Meyer, Krijn Reijnders, Francisco Rodríguez-Henríquez, Peter Schwabe, and Thom Wiggers. Optimizations and Practicality of High-Security CSIDH. *CiC*, 1(1):5, 2024.
- [CD23] Wouter Castryck and Thomas Decru. An Efficient Key Recovery Attack on SIDH. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part V*, volume 14008 of *LNCS*, pages 423–447. Springer, Cham, April 2023.
- [CDRH22] Jesús-Javier Chi-Domínguez and Francisco Rodríguez-Henríquez. Optimal strategies for CSIDH. *Advances in Mathematics of Communications*, 16(2):383–411, 2022.
- [CFG⁺21] Hao Cheng, Georgios Fotiadis, Johann Großschädl, Peter Y. A. Ryan, and Peter B. Rønne. Batching CSIDH Group Actions using AVX-512. *IACR TCHES*, 2021(4):618–649, 2021.
- [CFGR22] Hao Cheng, Georgios Fotiadis, Johann Großschädl, and Peter Y. A. Ryan. Highly Vectorized SIKE for AVX-512. *IACR TCHES*, 2022(2):41–68, 2022.

- [CHMR25] Fabio Campos, Andreas Hellenbrand, Michael Meyer, and Krijn Reijnders. dCTIDH: Fast & deterministic CTIDH. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(3):516–541, 2025.
- [CL84] H. Cohen and H. W. Lenstra. Heuristics on class groups of number fields. In Hendrik Jager, editor, *Number Theory Noordwijkerhout 1983*, pages 33–62, Berlin, Heidelberg, 1984. Springer Berlin Heidelberg.
- [CLM⁺18] Wouter Castryck, Tanja Lange, Chloe Martindale, Lorenz Panny, and Joost Renes. CSIDH: An Efficient Post-Quantum Commutative Group Action. In Thomas Peyrin and Steven Galbraith, editors, *ASIACRYPT 2018, Part III*, volume 11274 of *LNCS*, pages 395–427. Springer, Cham, December 2018.
- [Cou06] Jean-Marc Couveignes. Hard Homogeneous Spaces. Cryptology ePrint Archive, Report 2006/291, 2006.
- [CS18] Craig Costello and Benjamin Smith. Montgomery curves and their arithmetic. *Journal of Cryptographic Engineering*, 8(3):227–240, Sep 2018.
- [CSCJR22] Jorge Chávez-Saab, Jesús-Javier Chi-Domínguez, Samuel Jaques, and Francisco Rodríguez-Henríquez. The SQALE of CSIDH: sublinear Vélu quantum-resistant isogeny action with low exponents. *Journal of Cryptographic Engineering*, 12(3):349–368, September 2022.
- [DG19] Luca De Feo and Steven D. Galbraith. SeaSign: Compact Isogeny Signatures from Class Group Actions. In Yuval Ishai and Vincent Rijmen, editors, *EUROCRYPT 2019, Part III*, volume 11478 of *LNCS*, pages 759–789. Springer, Cham, May 2019.
- [DKS18] Luca De Feo, Jean Kieffer, and Benjamin Smith. Towards Practical Key Exchange from Ordinary Isogeny Graphs. In Thomas Peyrin and Steven Galbraith, editors, *ASIACRYPT 2018, Part III*, volume 11274 of *LNCS*, pages 365–394. Springer, Cham, December 2018.
- [DM20] Luca De Feo and Michael Meyer. Threshold schemes from isogeny assumptions. In Aggelos Kiayias, Markulf Kohlweiss, Petros Wallden, and Vassilis Zikas, editors, *PKC 2020: 23rd International Conference on Theory and Practice of Public Key Cryptography, Part II*, volume 12111 of *Lecture Notes in Computer Science*, pages 187–212, Edinburgh, UK, May 4–7, 2020. Springer, Cham, Switzerland.
- [FJP14] Luca De Feo, David Jao, and Jérôme Plût. Towards quantum-resistant cryptosystems from supersingular elliptic curve isogenies. *Journal of Mathematical Cryptology*, 8(3):209–247, 2014.
- [GdKQ⁺24] Phillip Gajland, Bor de Kock, Miguel Quaresma, Giulio Malavolta, and Peter Schwabe. SWOOSH: Efficient Lattice-Based Non-Interactive Key Exchange. In *USENIX Security 2024*. USENIX Association, August 2024.
- [HLKA20] Aaron Hutchinson, Jason T. LeGrow, Brian Koziel, and Reza Azarderakhsh. Further Optimizations of CSIDH: A Systematic Approach to Efficient Strategies, Permutations, and Bound Vectors. In Mauro Conti, Jianying Zhou, Emiliano Casalichio, and Angelo Spognardi, editors, *ACNS 20International Conference on Applied Cryptography and Network Security, Part I*, volume 12146 of *LNCS*, pages 481–501. Springer, Cham, October 2020.

- [HLM⁺25] Lucjan Hanzlik, Yi-Fu Lai, Marzio Mula, Eugenio Paracucchi, Daniel Slamanig, and Gang Tang. Tanuki: New frameworks for (concurrently secure) blind signatures from post-quantum group actions. In Goichiro Hanaoka and Bo-Yin Yang, editors, *Advances in Cryptology – ASIACRYPT 2025, Part IV*, volume 16248 of *Lecture Notes in Computer Science*, pages 35–69, Melbourne, VIC, Australia, December 8–12, 2025. Springer, Singapore, Singapore.
- [Hou25] Marc Houben. Deterministic Algorithms for Class Group Actions. In *CRYPTO 2025, Part I*, LNCS, pages 100–130. Springer, Cham, August 2025.
- [Hou26] Marc Houben. Efficient Post-quantum Commutative Group Actions from Orientations of Large Discriminant. In Goichiro Hanaoka and Bo-Yin Yang, editors, *Advances in Cryptology – ASIACRYPT 2025*, pages 141–173, Singapore, 2026. Springer Nature Singapore.
- [Int25] Intel Corporation. *Intel[®] Architecture Instruction Set Extensions and Future Features*, revision 60 edition, November 2025. Document Number: 319433-060.
- [Int26] Intel Corporation. *Intel[®] Advanced Vector Extensions 10.2 Architecture Specification*, revision 6.0 edition, January 2026. Document Number: 361050-006US.
- [JAC⁺22] David Jao, Reza Azarderakhsh, Matthew Campagna, Craig Costello, Luca De Feo, Basil Hess, Amir Jalali, Brian Koziel, Brian LaMacchia, Patrick Longa, Michael Naehrig, Joost Renes, Vladimir Soukharev, David Urbanik, Geovandro Pereira, Koray Karabina, and Aaron Hutchinson. SIKE. Technical report, National Institute of Standards and Technology, 2022. available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/round-4-submissions>.
- [JD11] David Jao and Luca De Feo. Towards Quantum-Resistant Cryptosystems from Supersingular Elliptic Curve Isogenies. In Bo-Yin Yang, editor, *Post-Quantum Cryptography - 4th International Workshop, PQCrypto 2011*, pages 19–34. Springer, Berlin, Heidelberg, November / December 2011.
- [KG19] Dusan Kostic and Shay Gueron. Using the New VPMADD Instructions for the New Post Quantum Key Encapsulation Mechanism SIKE. In *2019 IEEE 26th Symposium on Computer Arithmetic (ARITH)*, pages 215–218, 2019.
- [KLLQ23] Shuichi Katsumata, Yi-Fu Lai, Jason T. LeGrow, and Ling Qin. CSI-Otter: Isogeny-based (partially) blind signatures from the class group action with a twist. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023, Part III*, volume 14083 of *Lecture Notes in Computer Science*, pages 729–761, Santa Barbara, CA, USA, August 20–24, 2023. Springer, Cham, Switzerland.
- [Kup05] Greg Kuperberg. A subexponential-time quantum algorithm for the dihedral hidden subgroup problem. *SIAM Journal on Computing*, 35(1):170–188, 2005.
- [Kup13] Greg Kuperberg. Another subexponential-time quantum algorithm for the dihedral hidden subgroup problem. In *8th Conference on the Theory of Quantum Computation, Communication and Cryptography (TQC 2013)*, volume 22, pages 20–34, Dagstuhl, Germany, 2013. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik.

- [LGD21] Yi-Fu Lai, Steven D. Galbraith, and Cyprien Delpech de Saint Guilhem. Compact, efficient and UC-secure isogeny-based oblivious transfer. In Anne Canteaut and François-Xavier Standaert, editors, *Advances in Cryptology – EUROCRYPT 2021, Part I*, volume 12696 of *Lecture Notes in Computer Science*, pages 213–241, Zagreb, Croatia, October 17–21, 2021. Springer, Cham, Switzerland.
- [MCR19] Michael Meyer, Fabio Campos, and Steffen Reith. On Lions and Elligators: An Efficient Constant-Time Implementation of CSIDH. In Jintai Ding and Rainer Steinwandt, editors, *Post-Quantum Cryptography - 10th International Conference, PQCrypto 2019*, pages 307–325. Springer, Cham, 2019.
- [MMP⁺23] Luciano Maino, Chloe Martindale, Lorenz Panny, Giacomo Pope, and Benjamin Wesolowski. A Direct Key Recovery Attack on SIDH. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part V*, volume 14008 of *LNCS*, pages 448–471. Springer, Cham, April 2023.
- [MR18] Michael Meyer and Steffen Reith. A Faster Way to the CSIDH. In Debrup Chakraborty and Tetsu Iwata, editors, *INDOCRYPT 2018*, volume 11356 of *LNCS*, pages 137–152. Springer, Cham, December 2018.
- [NO24] Kohei Nakagawa and Hiroshi Onuki. QFESTA: Efficient algorithms and parameters for FESTA using quaternion algebras. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology – CRYPTO 2024, Part V*, volume 14924 of *Lecture Notes in Computer Science*, pages 75–106, Santa Barbara, CA, USA, August 18–22, 2024. Springer, Cham, Switzerland.
- [NOTT23] Kohei Nakagawa, Hiroshi Onuki, Atsushi Takayasu, and Tsuyoshi Takagi. l_1 -norm ball for csidh: Optimal strategy for choosing the secret key space. *Discrete Applied Mathematics*, 328:70–88, 2023.
- [OAYT19] Hiroshi Onuki, Yusuke Aikawa, Tsutomu Yamazaki, and Tsuyoshi Takagi. A Faster Constant-Time Algorithm of CSIDH Keeping Two Points. In Nuttapong Attrapadung and Takeshi Yagi, editors, *Advances in Information and Computer Security*, pages 23–33, Cham, 2019. Springer International Publishing.
- [OLHA18] Gabriell Orisaka, Julio César López-Hernández, and Diego F. Aranha. Finite Field Arithmetic Using AVX-512 For Isogeny-Based Cryptography. In *18th Brazilian Symposium on Information and Computer Systems Security (SBSeg)*, pages 49–56, 2018.
- [Rob23] Damien Robert. Breaking SIDH in Polynomial Time. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part V*, volume 14008 of *LNCS*, pages 472–503. Springer, Cham, April 2023.
- [RS06] Alexander Rostovtsev and Anton Stolbunov. PUBLIC-KEY CRYPTOSYSTEM BASED ON ISOGENIES. Cryptology ePrint Archive, Report 2006/145, 2006.
- [Sil09] J. H. Silverman. *The Arithmetic of Elliptic Curves, 2nd Edition*. Graduate Texts in Mathematics. Springer, 2009.
- [Vél71] Jacques Vélú. Isogénies entre courbes elliptiques. *C. R. Acad. Sci., Paris, Sér. A*, 273:238–241, 1971.
- [Voi21] John Voight. *Quaternion algebras*. Springer Graduate Texts in Mathematics series., 2021.